

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN ĐIỀU KHIỂN TỰ ĐỘNG



TIỂU LUẬN MÔN HỌC
— ĐO LƯỜNG ĐIỀU KHIỂN BẰNG MÁY TÍNH —

**XÂY DỰNG HỆ THỐNG GIÁM SÁT VÀ ĐIỀU KHIỂN THỜI
GIAN THỰC TỐC ĐỘ ĐỘNG CƠ DC QUA ETHERNET VÀ
PYTHON/SIMULINK**

GIẢNG VIÊN HƯỚNG DẪN:

TS. NGUYỄN TRỌNG TÀI

SINH VIÊN THỰC HIỆN:

Nguyễn Văn Tuấn Anh – 2310131

Hồ Công Thành – 2313111

Cao Chí Nguyên – 2312331

TP. HỒ CHÍ MINH, 2026

Mục lục

1	TỔNG QUAN ĐỀ TÀI	1
1.1	Lý do chọn đề tài	1
1.2	Mục tiêu đề tài	2
1.3	Đối tượng và phạm vi nghiên cứu	2
1.4	Sơ đồ tổng quan hệ thống	3
2	CƠ SỞ LÝ THUYẾT VỀ ETHERNET VÀ TCP/IP	4
2.1	Tổng quan về Ethernet	4
2.2	Mô hình TCP/IP	5
2.3	Cấu trúc Frame truyền của hệ thống	5
3	MẠCH NGUYÊN LÝ VÀ LAYOUT	7
3.1	Sơ đồ nguyên lí	7
3.2	PCB Layout	8
4	THIẾT KẾ FIRMWARE HỆ THỐNG	10
4.1	Tổng quan cấu trúc firmware	10
4.2	Bộ nhớ đệm lệnh và telemetry	12
4.3	Khởi tạo các module phần cứng	13
4.3.1	Khởi tạo encoder	13
4.3.2	Khởi tạo điều khiển động cơ	14
4.3.3	Khởi tạo Ethernet	16

4.4	Thiết kế vòng điều khiển thời gian thực trên Core 1	17
4.4.1	Timer ISR tối giản	17
4.4.2	Tạo task điều khiển và ghim vào Core 1	17
4.4.3	Đọc encoder và tính RPM	18
4.4.4	Cập nhật PWM trong task realtime	19
4.4.5	Bộ điều khiển PI	20
4.4.6	Luồng xử lý của task realtime	21
4.5	Task TCP Server trên Core 0	23
4.5.1	Tạo TCP server	23
4.5.2	TCP không trực tiếp điều khiển PWM	24
4.6	Giao thức frame truyền thông	25
4.7	Giám sát an toàn và timeout truyền thông	27
4.8	Tạo các task trong hàm app_main	28
4.9	Luồng hoạt động của firmware	29
5	THIẾT KẾ GIAO DIỆN QUẢN LÝ BẰNG PYTHON	31
5.1	Tổng quan giao diện Python	31
5.2	Định nghĩa giao thức và tham số giao tiếp	33
5.3	Tính CRC và đóng gói frame @SEN	34
5.4	Nhận và giải mã frame @REC	35
5.5	Luồng TCP Client RealtimeClient	36
5.6	Thiết kế lớp giao diện MotorGUI	38
5.7	Kết nối và ngắt kết nối ESP32	38
5.8	Điều khiển Manual PWM	39
5.9	Cấu hình bộ điều khiển PI trong firmware	40
5.10	Đọc telemetry và monitor tự động	41
5.11	Xử lý sự kiện từ luồng TCP	42
5.12	Cập nhật dữ liệu phản hồi lên giao diện	43

5.13	Vẽ biểu đồ realtime	44
5.14	Ghi log và đóng chương trình	44
6	TỔNG KẾT	46
6.1	Kết quả đạt được	46
6.2	Ý nghĩa của kiến trúc realtime cục bộ	47
6.3	Hạn chế	47
6.4	Hướng phát triển	47

Danh sách hình vẽ

2.1	Cấu trúc tổng quát	5
2.2	Cấu trúc frame truyền của Ethernet	5
2.3	Cấu trúc frame truyền của IP Package	6
2.4	Cấu trúc frame truyền của TCP segment	6
3.1	Sơ đồ kết nối chân của ESP32	7
3.2	Sơ đồ kết nối chân của module W5500	8
3.3	Sơ đồ kết nối chân để kết nối với L298N	8
3.4	Sơ đồ kết nối chân của L298N	8
3.5	Sơ đồ kết nối chân để kết nối với DC Servo	8
3.6	Hình 4.6: Sơ đồ kết nối chân của DC Servo	8
3.7	Hình 4.7: PCB Layout	8
3.8	Hình 4.8: 3D View	9
4.1	Môi trường phát triển firmware ESP32 sử dụng ESP-IDF trên Visual Studio Code	10
4.2	Sơ đồ cấu trúc firmware theo mô hình hai core: TCP ở Core 0, realtime control ở Core 1.	12
5.1	Tổng quan giao diện Python Tkinter điều khiển và giám sát motor	32
5.2	Giao diện được thiết kế	33

Danh sách bảng

Chương 1

TỔNG QUAN ĐỀ TÀI

1.1 Lý do chọn đề tài

Trong các hệ thống điều khiển và giám sát hiện đại, việc truyền dữ liệu giữa thiết bị điều khiển và máy tính giám sát đóng vai trò rất quan trọng. Các chuẩn giao tiếp truyền thống như UART, USB hoặc truyền thông không dây có thể đáp ứng tốt trong các ứng dụng đơn giản, tuy nhiên khi hệ thống yêu cầu khoảng cách truyền xa hơn, tốc độ ổn định hơn và khả năng kết nối trong mạng LAN thì Ethernet là một lựa chọn phù hợp.

Ethernet là chuẩn truyền thông mạng có dây được sử dụng phổ biến trong công nghiệp, phòng thí nghiệm, hệ thống đo lường và điều khiển. Ưu điểm của Ethernet là khả năng truyền dữ liệu ổn định, tốc độ cao, dễ mở rộng và có thể kết nối nhiều thiết bị trong cùng một mạng. Trên nền Ethernet, các giao thức TCP/IP cho phép các thiết bị giao tiếp với nhau thông qua địa chỉ IP và port, giúp việc xây dựng hệ thống điều khiển từ máy tính trở nên thuận tiện hơn.

ESP32 là một vi điều khiển có hiệu năng tốt, tích hợp nhiều ngoại vi như GPIO, ADC, PWM, SPI, UART, WiFi và Bluetooth. Tuy nhiên, ESP32 không có sẵn cổng Ethernet RJ45 trên các board phát triển thông dụng. Vì vậy, việc kết hợp ESP32 với module Ethernet W5500 thông qua giao tiếp SPI cho phép xây dựng một card giao tiếp Ethernet có khả năng nhận lệnh từ máy tính, xử lý dữ liệu và điều khiển thiết bị ngoại vi.

Trong đề tài này, nhóm thực hiện xây dựng một hệ thống card giao tiếp Ethernet sử dụng ESP32 và W5500. Hệ thống cho phép phần mềm trên máy tính giao tiếp với ESP32 thông qua mạng TCP/IP. Dữ liệu truyền nhận giữa máy tính và ESP32 được đóng gói theo frame cố định 12 byte với định dạng @SEN cho frame truyền đi và @REC cho frame phản hồi. Ngoài ra, hệ thống được ứng dụng để điều khiển động cơ DC có encoder, đọc

tốc độ phản hồi và triển khai bộ điều khiển PI thông qua firmware ESP32, giao diện Python hoặc các công cụ giám sát như Matlab/Simulink.

Vì vậy, đề tài có ý nghĩa trong việc kết hợp kiến thức về vi điều khiển, truyền thông Ethernet, TCP/IP, lập trình giao diện Python, thiết kế giao thức truyền thông và điều khiển hệ thống thực tế.

1.2 Mục tiêu đề tài

Mục tiêu chính của đề tài là thiết kế và xây dựng một hệ thống giao tiếp Ethernet sử dụng ESP32 và module W5500, cho phép máy tính truyền nhận dữ liệu với ESP32 thông qua giao thức TCP/IP.

Các mục tiêu cụ thể gồm:

Tìm hiểu mô hình TCP/IP và vai trò của Ethernet trong hệ thống truyền thông mạng.

Thiết kế card giao tiếp Ethernet sử dụng ESP32 kết hợp module W5500.

Cấu hình ESP32 giao tiếp với W5500 thông qua chuẩn SPI.

Xây dựng ESP32 đóng vai trò TCP Server để nhận kết nối từ máy tính.

Xây dựng phần mềm Python Tkinter đóng vai trò TCP Client để kết nối, gửi lệnh và nhận dữ liệu từ ESP32.

Thiết kế frame truyền thông cố định 12 byte với định dạng @SEN và @REC.

Xây dựng cơ chế kiểm tra dữ liệu bằng mã kiểm tra CRC/XOR.

Điều khiển các tín hiệu ngoại vi như LED, PWM, ADC và encoder.

Ứng dụng hệ thống vào điều khiển tốc độ động cơ DC có encoder.

Khảo sát khả năng kết nối với Matlab/Simulink hoặc VISA để phục vụ mô phỏng, giám sát và điều khiển.

1.3 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài là hệ thống truyền thông Ethernet giữa máy tính và vi điều khiển ESP32 thông qua module W5500. Trong đó, ESP32 đóng vai trò thiết bị điều khiển và thu thập dữ liệu, còn máy tính đóng vai trò giao diện giám sát, điều khiển và xử lý dữ liệu.

Phạm vi nghiên cứu của đề tài bao gồm:

Truyền thông Ethernet trong mạng LAN.

Giao thức TCP/IP giữa máy tính và ESP32.

Giao tiếp SPI giữa ESP32 và W5500.

Lập trình TCP Server trên ESP32.

Lập trình TCP Client trên Python Tkinter.

Thiết kế frame truyền thông cố định 12 byte.

Điều khiển PWM, đọc ADC, đọc encoder và điều khiển động cơ DC.

Ứng dụng bộ điều khiển PI trong điều khiển tốc độ động cơ.

Đề tài tập trung vào việc xây dựng hệ thống giao tiếp và điều khiển ở mức mô hình thực nghiệm. Các vấn đề chuyên sâu về thiết kế mạch in công nghiệp, bảo mật mạng, tối ưu thời gian thực ở mức hệ điều hành thời gian thực hoặc triển khai hệ thống công nghiệp quy mô lớn không thuộc phạm vi chính của đề tài.

1.4 Sơ đồ tổng quan hệ thống

Chương 2

CƠ SỞ LÝ THUYẾT VỀ ETHERNET VÀ TCP/IP

2.1 Tổng quan về Ethernet

Ethernet là công nghệ truyền thông mạng cục bộ có dây, thường được sử dụng trong các hệ thống mạng LAN. Ethernet quy định cách các thiết bị truyền dữ liệu qua môi trường vật lý như cáp xoắn đôi, đầu nối RJ45, switch mạng và card mạng.

Trong đề tài, ESP32 không có sẵn cổng Ethernet RJ45 trên board phát triển thông dụng, vì vậy cần sử dụng thêm module Ethernet W5500. Module W5500 đóng vai trò là bộ điều khiển Ethernet, giúp ESP32 có thể kết nối vào mạng LAN thông qua cổng RJ45. ESP32 giao tiếp với W5500 bằng chuẩn SPI.

Ethernet đảm nhiệm các chức năng chính sau:

Cung cấp môi trường truyền dữ liệu vật lý qua dây mạng RJ45.

Truyền dữ liệu trong phạm vi mạng LAN.

Sử dụng địa chỉ MAC để nhận diện thiết bị trong mạng cục bộ.

Đóng gói dữ liệu ở tầng thấp dưới dạng Ethernet frame.

Là nền tảng để các giao thức tầng trên như IP, TCP và UDP hoạt động.

Trong hệ thống thực tế, phía ESP32 sử dụng Ethernet thông qua W5500. Phía máy tính có thể kết nối vào router bằng Ethernet hoặc WiFi. Nếu máy tính dùng WiFi còn ESP32 dùng dây Ethernet, hai thiết bị vẫn có thể giao tiếp được nếu chúng nằm trong cùng một mạng LAN.

2.2 Mô hình TCP/IP

TCP/IP là bộ giao thức nền tảng được sử dụng trong truyền thông mạng. Mô hình TCP/IP thường được mô tả gồm 4 tầng chính:

Tầng ứng dụng - Application Layer

Python GUI

Tầng giao vận - Transport Layer

TCP/UDP

Tầng Internet - Internet Layer

IP

Tầng liên kết mạng- Network Access / Link Layer

Ethernet

Trong một số tài liệu, tầng thấp nhất có thể được gọi là tầng vật lý hoặc tầng liên kết mạng. Về bản chất, tầng này bao gồm các công nghệ truyền dữ liệu ở mức thấp như Ethernet hoặc WiFi.

Mô hình TCP/IP trong hệ thống của đề tài có thể biểu diễn như sau:

Khi Python GUI gửi một frame điều khiển xuống ESP32, frame này không đi trực tiếp trên dây Ethernet. Thay vào đó, dữ liệu được đóng gói qua nhiều lớp:

[Ethernet Frame [IP Packet [TCP Segment [@SEN ... #]]]]

Trong đó, @SEN ... # là dữ liệu ứng dụng do nhóm thiết kế, TCP đảm bảo dữ liệu được truyền ổn định, IP định tuyến dữ liệu tới đúng địa chỉ thiết bị, còn Ethernet truyền dữ liệu qua mạng vật lý.

2.3 Cấu trúc Frame truyền của hệ thống

Hình 2.1: Cấu trúc tổng quát

Hình 2.2: Cấu trúc frame truyền của Ethernet

Hình 2.3: Cấu trúc frame truyền của IP Package

Hình 2.4: Cấu trúc frame truyền của TCP segment

Chương 3

MẠCH NGUYÊN LÝ VÀ LAYOUT

3.1 Sơ đồ nguyên lí

Hình 3.1: Sơ đồ kết nối chân của ESP32

Cụm giao tiếp SPI với khối Ethernet W5500 (MO, MI, SCK, CS)

Các chân được chọn: IO23 (MOSI), IO19 (MISO), IO18 (SCK), IO5 (CS).

Lý do: Đây là cụm chân mặc định của bộ VSPI (Hardware SPI) trên ESP32.

ESP32 có thể map SPI ra bất kỳ chân nào thông qua GPIO Matrix (phần mềm), nhưng việc sử dụng cụm chân Hardware SPI mặc định giúp đường truyền đạt tốc độ tối đa (có thể lên tới 80MHz) mà không bị độ trễ (latency).

Thư viện `esp_eth` mặc định cũng sẽ tìm kiếm các chân này đầu tiên, giúp code của bạn gọn gàng và ít lỗi khởi tạo nhất.

Cụm điều khiển động cơ L298N (IN1, IN2, ENA)

Các chân được chọn: IO26 (IN1), IO25 (IN2), IO27 (ENA).

Lý do: Bộ điều khiển băm xung (LEDC) của ESP32 có thể xuất PWM ở nhiều chân, nhưng IO25, 26, 27 là các chân GPIO "sạch". Nghĩa là chúng không tham gia vào quá trình khởi động (Boot) của chip.

Khi bạn vừa cấp nguồn cho ESP32, các chân này sẽ nằm im ở mức LOW. Điều này cực kỳ an toàn cho mạch công suất L298N, giúp động cơ không bị giật hay tự động rô lên trong lúc chờ chip nạp code xong.

Cụm đọc tín hiệu đếm Encoder (Phase A, Phase B)

Các chân được chọn: IO32, IO33.

Lý do: Khối Pulse Counter (PCNT) cần những chân đọc tín hiệu đầu vào có độ ổn định cao. IO32 và IO33 thuộc tập hợp chân ADC1, được thiết kế với trở kháng đầu vào rất tốt.

So với các chân GPIO từ 34 đến 39 (vốn chỉ có thể làm Input và thiếu điện trở kéo nội bộ), thì IO32 và IO33 là các chân đa năng, có hỗ trợ Pull-up/Pull-down nội bộ (Internal Resistors), giúp tín hiệu xung vuông từ Encoder đưa về được "vuông vức" và ít nhiễu hơn.

Hình 3.2: Sơ đồ kết nối chân của module W5500

Hình 3.3: Sơ đồ kết nối chân để kết nối với L298N

Hình 3.4: Sơ đồ kết nối chân của L298N

Hình 3.5: Sơ đồ kết nối chân để kết nối với DC Servo

Hình 3.6: Hình 4.6: Sơ đồ kết nối chân của DC Servo

Vì lý do không thể kiểm được thư viện của module L298N và động cơ DC Servo có tích hợp Encoder, nên khi vẽ mạch PCB thì ta chỉ có thể kéo ra các chân của header.

3.2 PCB Layout

Hình 3.7: Hình 4.7: PCB Layout

Hình 3.8: Hình 4.8: 3D View

Chương 4

THIẾT KẾ FIRMWARE HỆ THỐNG

4.1 Tổng quan cấu trúc firmware

Firmware của hệ thống được xây dựng trên nền tảng ESP-IDF và chạy trên vi điều khiển ESP32. Chương trình có nhiệm vụ giao tiếp Ethernet TCP/IP với phần mềm máy tính, nhận lệnh cấu hình, điều khiển động cơ DC thông qua module L298N, đọc encoder và thực hiện vòng điều khiển PI ngay trên ESP32.



Hình 4.1: Môi trường phát triển firmware ESP32 sử dụng ESP-IDF trên Visual Studio Code

Khác với cách điều khiển trong đó máy tính đọc tốc độ rồi gửi PWM liên tục xuống vi điều khiển, firmware hiện tại được tổ chức theo kiến trúc điều khiển cục bộ trên ESP32. Ethernet TCP chỉ đóng vai trò nhận lệnh, cấu hình tham số và giám sát trạng thái; vòng

điều khiển tốc độ không phụ thuộc vào chu kỳ truyền thông của PC.

ESP32 được khai thác theo cấu trúc hai lõi:

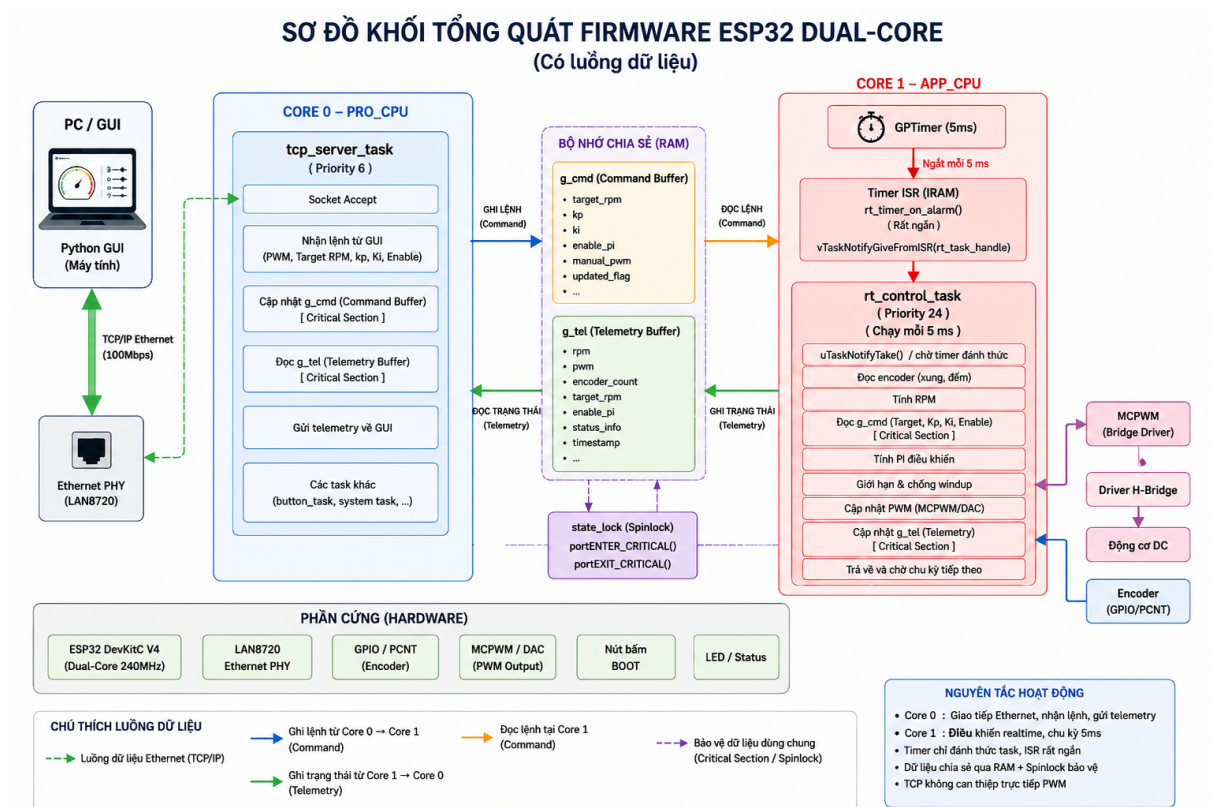
- **Core 0 – PRO CPU:** chạy `tcp_server_task`. Task này xử lý socket TCP, nhận frame @SEN, kiểm tra CRC, giải mã lệnh và cập nhật bộ đếm lệnh `g_cmd`. Task mạng không trực tiếp xuất PWM ra động cơ.
- **Core 1 – APP CPU:** chạy `rt_timer_setup_task`, `rt_control_task` và `button_task`. Trong đó `rt_control_task` là task ưu tiên cao nhất, chịu trách nhiệm đọc encoder, tính RPM, xử lý manual PWM hoặc PI, kiểm tra timeout và cập nhật PWM thật cho motor.

Việc tách hai lõi giúp phần truyền thông TCP, vốn có độ trễ và jitter, không nằm trong đường điều khiển trực tiếp. Các lệnh từ PC được ghi vào vùng nhớ chia sẻ, còn task điều khiển thời gian thực đọc snapshot lệnh và quyết định đầu ra PWM ở chu kỳ cố định.

```
#if CONFIG_FREERTOS_UNICORE
#define RT_CORE_ID 0
#define TCP_CORE_ID 0
#else
#define RT_CORE_ID APP_CPU_NUM
#define TCP_CORE_ID PRO_CPU_NUM
#endif

#define RT_CONTROL_PRIORITY 24
#define TCP_TASK_PRIORITY 6
#define BUTTON_TASK_PRIORITY 5
```

Nếu firmware được build ở chế độ một lõi, cả hai task sẽ chạy trên core 0. Nếu ESP32 chạy ở chế độ hai lõi, `RT_CORE_ID` được ánh xạ sang `APP_CPU_NUM` và `TCP_CORE_ID` được ánh xạ sang `PRO_CPU_NUM`. Do đó, cấu trúc thực thi tương ứng là Core 1 cho điều khiển và Core 0 cho truyền thông.



Hình 4.2: Sơ đồ cấu trúc firmware theo mô hình hai core: TCP ở Core 0, realtime control ở Core 1.

4.2 Bộ nhớ đệm lệnh và telemetry

Để tách riêng task mạng và task điều khiển, firmware sử dụng hai cấu trúc dữ liệu chia sẻ. Cấu trúc `control_command_t` chứa lệnh mới nhất từ PC, còn `telemetry_t` chứa dữ liệu phản hồi để PC đọc lại.

```
typedef struct {
    volatile int manual_pwm;
    volatile int target_rpm;
    volatile int kp_x100;
    volatile int ki_x100;
    volatile bool pi_enabled;
    volatile bool manual_pwm_pending;
    volatile int64_t last_command_time_us;
} control_command_t;

typedef struct {
```

```

    int rpm;
    int pwm;
    int target_rpm;
    uint32_t rx_frames;
    uint32_t deadline_misses;
    int64_t max_rt_exec_us;
    uint32_t fault_flags;
} telemetry_t;

```

Các biến dùng chung được bảo vệ bằng critical section ngắn:

```

static portMUX_TYPE state_lock = portMUX_INITIALIZER_UNLOCKED;

static control_command_t g_cmd = {
    .kp_x100 = 50,
    .ki_x100 = 10,
};

static telemetry_t g_tel;

```

Nhờ cách tổ chức này, `tcp_server_task` chỉ cập nhật yêu cầu điều khiển, còn `rt_control_task` là nơi duy nhất quyết định trạng thái motor. Điều này làm giảm khả năng mạng TCP gây nhiễu trực tiếp đến PWM.

4.3 Khởi tạo các module phần cứng

4.3.1 Khởi tạo encoder

Encoder được dùng để đo tốc độ thực tế của động cơ DC. Firmware sử dụng ngoại vi PCNT của ESP32 để đếm xung encoder trên chân GPIO32. Mỗi cạnh xung làm bộ đếm tăng lên một đơn vị. Trong thiết kế hiện tại chỉ sử dụng một kênh xung, vì vậy hệ thống đo tốc độ quay nhưng chưa xác định chiều quay.

```

#define PULSES_PER_REV 250

static void init_encoder(void)
{
    pcnt_unit_config_t unit_config = {

```

```

        .high_limit = 32767,
        .low_limit = -32768,
    };
    ESP_ERROR_CHECK(pcmt_new_unit(&unit_config, &pcmt_unit));

    pcmt_chan_config_t chan_config = {
        .edge_gpio_num = 32,
        .level_gpio_num = -1,
    };
    pcmt_channel_handle_t pcmt_chan = NULL;
    ESP_ERROR_CHECK(pcmt_new_channel(pcmt_unit, &chan_config, &pcmt_chan));
    ESP_ERROR_CHECK(pcmt_channel_set_edge_action(
        pcmt_chan,
        PCMT_CHANNEL_EDGE_ACTION_HOLD,
        PCMT_CHANNEL_EDGE_ACTION_INCREASE));
    ESP_ERROR_CHECK(pcmt_unit_enable(pcmt_unit));
    ESP_ERROR_CHECK(pcmt_unit_clear_count(pcmt_unit));
    ESP_ERROR_CHECK(pcmt_unit_start(pcmt_unit));
}

```

Thông số PULSES_PER_REV = 250 được dùng để đổi số xung đo được sang tốc độ vòng/phút. Nếu encoder thực tế có số xung trên vòng khác, cần thay đổi thông số này để kết quả RPM chính xác.

4.3.2 Khởi tạo điều khiển động cơ

Động cơ DC được điều khiển thông qua driver L298N. ESP32 dùng hai chân GPIO26 và GPIO25 để điều khiển chiều quay, đồng thời dùng GPIO27 làm chân PWM điều chỉnh tốc độ.

```

#define MOTOR_IN1_PIN 26
#define MOTOR_IN2_PIN 25
#define MOTOR_ENA_PIN 27

#define MOTOR_PWM_MODE LEDC_LOW_SPEED_MODE
#define MOTOR_PWM_TIMER LEDC_TIMER_0
#define MOTOR_PWM_CHANNEL LEDC_CHANNEL_0
#define MOTOR_PWM_DUTY_RES LEDC_TIMER_8_BIT

```

```
#define MOTOR_PWM_FREQ_HZ 5000
#define MOTOR_PWM_MAX_DUTY 255
```

PWM được tạo bằng ngoại vi LEDC của ESP32, tần số 5 kHz và độ phân giải 8 bit. Vì vậy giá trị duty hợp lệ nằm trong khoảng từ 0 đến 255.

```
static void init_motor(void)
{
    gpio_config_t motor_gpio_conf = {
        .pin_bit_mask = (1ULL << MOTOR_IN1_PIN) | (1ULL << MOTOR_IN2_PIN),
        .mode = GPIO_MODE_OUTPUT,
        .pull_up_en = GPIO_PULLUP_DISABLE,
        .pull_down_en = GPIO_PULLEDOWN_DISABLE,
        .intr_type = GPIO_INTR_DISABLE,
    };
    ESP_ERROR_CHECK(gpio_config(&motor_gpio_conf));

    ledc_timer_config_t ledc_timer = {
        .speed_mode = MOTOR_PWM_MODE,
        .timer_num = MOTOR_PWM_TIMER,
        .duty_resolution = MOTOR_PWM_DUTY_RES,
        .freq_hz = MOTOR_PWM_FREQ_HZ,
        .clk_cfg = LEDC_AUTO_CLK,
    };
    ESP_ERROR_CHECK(ledc_timer_config(&ledc_timer));

    ledc_channel_config_t ledc_channel = {
        .speed_mode = MOTOR_PWM_MODE,
        .channel = MOTOR_PWM_CHANNEL,
        .timer_sel = MOTOR_PWM_TIMER,
        .intr_type = LEDC_INTR_DISABLE,
        .gpio_num = MOTOR_ENA_PIN,
        .duty = 0,
        .hpoint = 0,
    };
    ESP_ERROR_CHECK(ledc_channel_config(&ledc_channel));
    set_motor_output_rt(0);
}
```

Trong firmware, chiều quay thuận được đặt bằng $IN1 = 1$, $IN2 = 0$. Khi PWM bằng 0, cả hai chân điều khiển chiều được đưa về mức 0 để dừng động cơ.

4.3.3 Khởi tạo Ethernet

Khối Ethernet được khởi tạo bằng hàm `example_eth_init()` của ESP-IDF. Khi chỉ có một cổng Ethernet, firmware đặt IP tĩnh cho ESP32 là 192.168.2.177, gateway là 192.168.2.1 và netmask là 255.255.255.0. TCP server lắng nghe tại port 5000.

```
#define PORT 5000

ESP_ERROR_CHECK(example_eth_init(&eth_handles, &eth_port_cnt));
ESP_ERROR_CHECK(esp_netif_init());
ESP_ERROR_CHECK(esp_event_loop_create_default());

esp_netif_config_t cfg = ESP_NETIF_DEFAULT_ETH();
eth_netifs[0] = esp_netif_new(&cfg);
eth_netif_glues[0] = esp_eth_new_netif_glue(eth_handles[0]);
ESP_ERROR_CHECK(esp_netif_attach(eth_netifs[0], eth_netif_glues[0]));
ESP_ERROR_CHECK(esp_netif_dhcpc_stop(eth_netifs[0]));

esp_netif_ip_info_t ip_info;
IP4_ADDR(&ip_info.ip, 192, 168, 2, 177);
IP4_ADDR(&ip_info.gw, 192, 168, 2, 1);
IP4_ADDR(&ip_info.netmask, 255, 255, 255, 0);
ESP_ERROR_CHECK(esp_netif_set_ip_info(eth_netifs[0], &ip_info));
```

Firmware cũng đăng ký các event handler để theo dõi trạng thái Ethernet. Khi Ethernet bị ngắt hoặc dừng, hệ thống gọi hàm dừng an toàn để yêu cầu motor về PWM bằng 0.

4.4 Thiết kế vòng điều khiển thời gian thực trên Core 1

4.4.1 Timer ISR tối giản

Firmware sử dụng GPTimer tạo chu kỳ cơ sở $RT_PERIOD_MS = 5\text{ ms}$. Tuy nhiên, ISR của timer không xử lý PI, không đọc encoder và không cập nhật PWM. ISR chỉ đánh thức task điều khiển thông qua cơ chế task notification của FreeRTOS.

```
#define RT_PERIOD_MS 5
#define RT_DEADLINE_BUDGET_US 500
#define PI_DIVIDER 10
#define PI_PERIOD_MS (RT_PERIOD_MS * PI_DIVIDER)

static bool IRAM_ATTR rt_timer_on_alarm(gptimer_handle_t timer,
                                         const gptimer_alarm_event_data_t *edata,
                                         void *user_ctx)
{
    (void)timer;
    (void)edata;
    (void)user_ctx;

    BaseType_t high_task_woken = pdFALSE;
    if (rt_task_handle != NULL) {
        vTaskNotifyGiveFromISR(rt_task_handle, &high_task_woken);
    }
    return high_task_woken == pdTRUE;
}
```

Cách làm này giúp ISR rất ngắn, giảm thời gian khóa ngắt và làm cho chu kỳ điều khiển ổn định hơn. Toàn bộ logic nặng được chuyển sang `rt_control_task` chạy ở mức task nhưng có ưu tiên cao.

4.4.2 Tạo task điều khiển và ghim vào Core 1

Task thiết lập timer tạo task điều khiển realtime và task đọc nút BOOT, sau đó cấu hình GPTimer chạy tự động.

```
xTaskCreatePinnedToCore(
    rt_control_task,
    "rt_control",
    4096,
    NULL,
    RT_CONTROL_PRIORITY,
    &rt_task_handle,
    RT_CORE_ID);
```

```
xTaskCreatePinnedToCore(
    button_task,
    "button_task",
    2048,
    NULL,
    BUTTON_TASK_PRIORITY,
    NULL,
    RT_CORE_ID);
```

Như vậy, cả task điều khiển và task kiểm tra nút dừng đều chạy trên core realtime. Trong đó `rt_control_task` có priority cao hơn nên luôn được ưu tiên xử lý khi GPTimer đánh thức.

4.4.3 Đọc encoder và tính RPM

Trong chu kỳ PI, firmware đọc số xung đã đếm được trong PCNT, xóa bộ đếm và tính RPM theo thời gian lấy mẫu. Vì chu kỳ PI hiện tại bằng `PI_PERIOD_MS = 50 ms`, tốc độ được tính từ số xung trong khoảng 50 ms.

```
static int IRAM_ATTR sample_rpm_isr(int period_ms)
{
    int pulse_count = 0;
    if (pcnt_unit_get_count(pcnt_unit, &pulse_count) != ESP_OK) {
        return 0;
    }
    (void)pcnt_unit_clear_count(pcnt_unit);

    int rpm = (pulse_count * 60000) / (PULSES_PER_REV * period_ms);
```



```

    if (rpm < 0) {
        rpm = 0;
    }
    return rpm;
}

```

Công thức tính RPM là:

$$\text{RPM} = \text{pulse_count} * 60000 / (\text{PULSES_PER_REV} * \text{period_ms})$$

Trong đó `pulse_count` là số xung đếm được, `PULSES_PER_REV` là số xung trên một vòng và `period_ms` là thời gian lấy mẫu tính theo mili giây.

4.4.4 Cập nhật PWM trong task realtime

Firmware dùng hàm `set_motor_output_rt()` để cập nhật đầu ra motor. Hàm này được gọi từ task realtime, không gọi trực tiếp từ task TCP. Giá trị PWM được giới hạn về khoảng 0 đến 255 trước khi ghi xuống LEDC.

```

static int IRAM_ATTR clamp_pwm(int pwm)
{
    if (pwm < 0) {
        return 0;
    }
    if (pwm > MOTOR_PWM_MAX_DUTY) {
        return MOTOR_PWM_MAX_DUTY;
    }
    return pwm;
}

static void set_motor_output_rt(int pwm)
{
    pwm = clamp_pwm(pwm);

    if (pwm == 0) {
        gpio_set_level(MOTOR_IN1_PIN, 0);
        gpio_set_level(MOTOR_IN2_PIN, 0);
    } else {

```

```

        gpio_set_level(MOTOR_IN1_PIN, 1);
        gpio_set_level(MOTOR_IN2_PIN, 0);
    }

    (void)ledc_set_duty(MOTOR_PWM_MODE, MOTOR_PWM_CHANNEL, pwm);
    (void)ledc_update_duty(MOTOR_PWM_MODE, MOTOR_PWM_CHANNEL);

    portENTER_CRITICAL(&state_lock);
    g_tel.pwm = pwm;
    portEXIT_CRITICAL(&state_lock);
}

```

Điểm quan trọng là task TCP chỉ ghi yêu cầu vào `g_cmd`. Việc ghi GPIO và LEDC thật được tập trung tại task realtime, giúp đường điều khiển motor có tính xác định cao hơn.

4.4.5 Bộ điều khiển PI

Bộ điều khiển PI sử dụng hệ số K_p và K_i ở dạng số nguyên nhân 100. Ví dụ, `kp_x100 = 50` tương ứng với $K_p = 0.50$. Thành phần tích phân được giới hạn bởi `INTEGRAL_LIMIT` để giảm hiện tượng tích phân bão hòa.

```

#define MOTOR_PWM_MIN_RUN 80
#define INTEGRAL_LIMIT 20000

static int pi_step_common(int target, int rpm, int kp_x100, int ki_x100)
{
    int error = target - rpm;
    s_pi_integral += error;

    if (s_pi_integral > INTEGRAL_LIMIT) {
        s_pi_integral = INTEGRAL_LIMIT;
    } else if (s_pi_integral < -INTEGRAL_LIMIT) {
        s_pi_integral = -INTEGRAL_LIMIT;
    }

    int proportional = kp_x100 * error;
    int integral = (ki_x100 * s_pi_integral * PI_PERIOD_MS) / 1000;
}

```

```

int pwm = (proportional + integral) / 100;

if (pwm > MOTOR_PWM_MAX_DUTY) {
    pwm = MOTOR_PWM_MAX_DUTY;
} else if (pwm < 0) {
    pwm = 0;
}

if (pwm > 0 && pwm < MOTOR_PWM_MIN_RUN) {
    pwm = MOTOR_PWM_MIN_RUN;
}

return pwm;
}

```

Ngoài PI, firmware còn có cơ chế kick-start. Khi motor đang đứng yên, PWM hiện tại bằng 0 và có yêu cầu chạy PI, hệ thống cấp `MOTOR_KICK_START_PWM` trong một khoảng ngắn để thắng ma sát ban đầu. Sau đó, PWM được tính bằng PI dựa trên RPM thực tế.

4.4.6 Luồng xử lý của task realtime

`rt_control_task` chờ notification từ GPTimer. Sau mỗi lần được đánh thức, task đo thời gian bắt đầu, xử lý timeout truyền thông, xử lý manual PWM hoặc PI, cập nhật telemetry và ghi nhận deadline miss nếu thời gian xử lý vượt quá ngân sách.

```

static void rt_control_task(void *arg)
{
    static int pi_countdown = PI_DIVIDER;
    static int kick_ticks_remaining = 0;
    static int current_rpm = 0;

    while (1) {
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);

        int64_t start_us = esp_timer_get_time();

        pi_countdown--;
        if (pi_countdown <= 0) {
            current_rpm = sample_rpm_isr(PI_PERIOD_MS);

```

```

    }

    control_command_t cmd;
    portENTER_CRITICAL(&state_lock);
    cmd = g_cmd;
    portEXIT_CRITICAL(&state_lock);

    bool command_timeout = false;
    if (cmd.last_command_time_us > 0) {
        command_timeout =
            (start_us - cmd.last_command_time_us) > (COMMAND_TIMEOUT_MS *
                1000LL);
    }

    if (command_timeout) {
        s_pi_integral = 0;
        set_motor_output_rt(0);
    } else if (cmd.manual_pwm_pending) {
        portENTER_CRITICAL(&state_lock);
        g_cmd.manual_pwm_pending = false;
        portEXIT_CRITICAL(&state_lock);

        set_motor_output_rt(cmd.manual_pwm);
        kick_ticks_remaining = 0;
        s_pi_integral = 0;
    } else if (pi_countdown <= 0) {
        pi_countdown = PI_DIVIDER;
        if (cmd.pi_enabled) {
            set_motor_output_rt(
                pi_step_common(cmd.target_rpm, current_rpm,
                    cmd.kp_x100, cmd.ki_x100));
        }
    }
}
}
}

```

Đoạn code trên được rút gọn để thể hiện luồng chính. Trong firmware đầy đủ, task còn xử lý trường hợp target RPM bằng 0, kick-start, fault flag, thống kê thời gian thực

thi lớn nhất và số lần vượt deadline.

4.5 Task TCP Server trên Core 0

4.5.1 Tạo TCP server

Task TCP tạo socket, bind vào port 5000, listen và accept kết nối từ phần mềm PC. Sau khi client kết nối, firmware nhận chính xác từng frame 12 byte, kiểm tra hợp lệ, xử lý lệnh và gửi lại frame phản hồi.

```
static void tcp_server_task(void *pvParameters)
{
    int listen_sock = socket(AF_INET, SOCK_STREAM, IPPROTO_IP);

    struct sockaddr_in addr = {
        .sin_family = AF_INET,
        .sin_port = htons(PORT),
        .sin_addr.s_addr = htonl(INADDR_ANY),
    };

    bind(listen_sock, (struct sockaddr *)&addr, sizeof(addr));
    listen(listen_sock, 1);

    while (1) {
        int sock = accept(listen_sock, NULL, NULL);
        int nodelay = 1;
        setsockopt(sock, IPPROTO_TCP, TCP_NODELAY, &nodelay, sizeof(nodelay));

        while (1) {
            uint8_t rx[FRAME_LEN] = {0};
            uint8_t tx[FRAME_LEN] = {0};
            int len = recv_exact(sock, rx, sizeof(rx));
            if (len <= 0) {
                stop_motor_safely();
                break;
            }

            uint16_t value = 0;
```

```

uint8_t status = STATUS_OK;
uint8_t error = 0;

if (validate_request(rx, &error)) {
    handle_command(rx, &value, &status, &error);
} else {
    status = STATUS_ERROR;
    value = 0;
}

build_response(tx, rx[4], value, status, error, rx[9]);
send(sock, tx, sizeof(tx), 0);
}

shutdown(sock, 0);
close(sock);
}
}

```

Khi mất kết nối TCP, firmware gọi `stop_motor_safely()`. Hàm này không trực tiếp ghi PWM, mà đặt yêu cầu manual PWM bằng 0 để task realtime xử lý ở chu kỳ kế tiếp.

4.5.2 TCP không trực tiếp điều khiển PWM

Khi PC gửi lệnh ghi PWM, task TCP chỉ cập nhật `g_cmd`. Cờ `manual_pwm_pending` báo cho task realtime biết có lệnh PWM mới cần áp dụng.

```

case CMD_WRITE_PWM: {
    int pwm = request_value > MOTOR_PWM_MAX_DUTY ?
        MOTOR_PWM_MAX_DUTY : request_value;
    portENTER_CRITICAL(&state_lock);
    g_cmd.pi_enabled = false;
    g_cmd.manual_pwm = pwm;
    g_cmd.manual_pwm_pending = true;
    g_cmd.last_command_time_us = esp_timer_get_time();
    portEXIT_CRITICAL(&state_lock);
    *value = pwm;
    break;
}

```

```
}
```

Đối với chế độ PI, PC gửi tốc độ đặt, hệ số K_p , K_i và lệnh bật PI. Task TCP chỉ ghi các tham số này vào `g_cmd`; vòng điều khiển PI thật vẫn chạy trong `rt_control_task`.

```
case CMD_SET_TARGET_RPM:
    set_target_rpm(request_value);
    break;

case CMD_SET_KP:
    set_gains(request_value, st_cmd.ki_x100);
    break;

case CMD_SET_KI:
    set_gains(st_cmd.kp_x100, request_value);
    break;

case CMD_PI_ENABLE:
    set_pi_enabled(request_value != 0);
    *value = request_value != 0 ? 1 : 0;
    break;
```

4.6 Giao thức frame truyền thông

Dữ liệu giữa PC và ESP32 được đóng gói theo frame cố định 12 byte. Frame từ PC xuống ESP32 có header `@SEN`, frame phản hồi từ ESP32 về PC có header `@REC`. Byte cuối cùng của frame là ký tự `#`.

Byte	Frame gửi @SEN	Ý nghĩa
0	@	Ký tự bắt đầu frame
1–3	SEN	Header frame gửi từ PC xuống ESP32
4	CMD	Mã lệnh
5	DATA_H	Byte cao của dữ liệu 16 bit
6	DATA_L	Byte thấp của dữ liệu 16 bit
7	OPT1	Tùy chọn 1, dùng làm selector khi đọc status
8	OPT2	Tùy chọn 2
9	SEQ	Số thứ tự frame
10	CRC	Checksum XOR
11	#	Ký tự kết thúc frame

Byte	Frame phản hồi @REC	Ý nghĩa
0	@	Ký tự bắt đầu frame
1–3	REC	Header frame phản hồi từ ESP32 về PC
4	CMD	Mã lệnh được phản hồi
5	DATA_H	Byte cao của dữ liệu trả về
6	DATA_L	Byte thấp của dữ liệu trả về
7	STATUS	Trạng thái xử lý, 0 là OK
8	ERROR	Mã lỗi nếu có
9	SEQ	Số thứ tự frame, giữ nguyên từ request
10	CRC	Checksum XOR
11	#	Ký tự kết thúc frame

Checksum được tính bằng phép XOR 10 byte đầu của frame:

```
static uint8_t calc_crc(const uint8_t *frame)
{
    uint8_t crc = 0;
    for (int i = 0; i < 10; i++) {
        crc ^= frame[i];
    }
    return crc;
}
```


Khi nhận frame, firmware kiểm tra header, byte kết thúc và CRC. Nếu frame sai định dạng hoặc sai CRC, ESP32 trả về trạng thái lỗi và không cập nhật lệnh điều khiển.

```
static bool validate_request(const uint8_t *rx, uint8_t *error)
{
    if (rx[0] != '@' || rx[1] != 'S' || rx[2] != 'E' ||
        rx[3] != 'N' || rx[11] != '#') {
        *error = ERR_BAD_FRAME;
        return false;
    }
    if (calc_crc(rx) != rx[10]) {
        *error = ERR_BAD_CRC;
        return false;
    }
    return true;
}
```

Các mã lệnh chính gồm ghi PWM thủ công, đọc RPM, đọc PWM hiện tại, đặt target RPM, đặt hệ số Kp, đặt hệ số Ki, bật/tắt PI và đọc các trạng thái như fault flag, deadline miss, thời gian thực thi realtime lớn nhất và số frame đã nhận.

4.7 Giám sát an toàn và timeout truyền thông

Firmware có cơ chế timeout truyền thông áp dụng cho cả manual PWM và PI. Nếu trong thời gian COMMAND_TIMEOUT_MS không có lệnh hợp lệ mới từ PC, task realtime sẽ xóa tích phân PI, tắt PI, đưa PWM về 0 và đặt fault FAULT_COMM_TIMEOUT.

```
#define COMMAND_TIMEOUT_MS 1000
#define FAULT_NONE          0x0000
#define FAULT_COMM_TIMEOUT 0x0001

if (command_timeout) {
    s_pi_integral = 0;
    kick_ticks_remaining = 0;
    pi_countdown = PI_DIVIDER;
```

```

portENTER_CRITICAL(&state_lock);
g_cmd.pi_enabled = false;
g_cmd.manual_pwm_pending = false;
g_cmd.manual_pwm = 0;
portEXIT_CRITICAL(&state_lock);

set_motor_output_rt(0);
fault_flags |= FAULT_COMM_TIMEOUT;
}

```

Ngoài timeout truyền thông, nút BOOT trên GPIO0 được kiểm tra trong `button_task`. Khi nút được nhấn, firmware gọi `stop_motor_safely()` để yêu cầu dừng motor.

```

static void button_task(void *arg)
{
    while (1) {
        if (gpio_get_level(BOOT_BUTTON_PIN) == 0) {
            stop_motor_safely();
        }
        vTaskDelay(pdMS_TO_TICKS(20));
    }
}

```

Việc đọc nút nhấn được đưa ra khỏi ISR giúp timer ISR luôn ngắn, không có delay và không chứa xử lý chặn.

4.8 Tạo các task trong hàm `app_main`

Trong `app_main()`, firmware lần lượt khởi tạo nút BOOT, encoder, motor, task realtime, Ethernet và task TCP. Task realtime được tạo trước để sẵn sàng điều khiển motor, sau đó Ethernet được start và TCP server bắt đầu lắng nghe kết nối từ PC.

```

void app_main(void)
{
    gpio_reset_pin(BOOT_BUTTON_PIN);
    gpio_set_direction(BOOT_BUTTON_PIN, GPIO_MODE_INPUT);
    gpio_set_pull_mode(BOOT_BUTTON_PIN, GPIO_PULLUP_ONLY);
}

```

```

init_encoder();
init_motor();

portENTER_CRITICAL(&state_lock);
g_cmd.last_command_time_us = esp_timer_get_time();
portEXIT_CRITICAL(&state_lock);

xTaskCreatePinnedToCore(
    rt_timer_setup_task,
    "rt_timer_setup",
    4096,
    NULL,
    RT_CONTROL_PRIORITY - 1,
    NULL,
    RT_CORE_ID);

/* Ethernet init, netif attach, event register, esp_eth_start */

xTaskCreatePinnedToCore(
    tcp_server_task,
    "tcp_server",
    4096,
    NULL,
    TCP_TASK_PRIORITY,
    NULL,
    TCP_CORE_ID);
}

```

Đoạn tạo task trên thể hiện rõ cấu trúc hai core của firmware: phần realtime được ghim vào RT_CORE_ID, còn phần TCP được ghim vào TCP_CORE_ID. Với ESP32 dual-core, điều này tương ứng với Core 1 cho điều khiển và Core 0 cho mạng.

4.9 Luồng hoạt động của firmware

Khi ESP32 được cấp nguồn, chương trình khởi tạo encoder PCNT, PWM LEDC, GPIO điều khiển motor và Ethernet. Sau đó firmware tạo task realtime trên Core 1, cấu hình GPTimer chu kỳ 5 ms, rồi tạo TCP server trên Core 0.

Trong quá trình chạy, GPTimer định kỳ đánh thức `rt_control_task`. Task này đọc snapshot lệnh từ `g_cmd`, kiểm tra timeout, đọc RPM theo chu kỳ PI, tính PWM nếu PI đang bật hoặc áp dụng PWM thủ công nếu có lệnh mới. Sau khi cập nhật motor, task ghi dữ liệu giám sát vào `g_tel`.

Song song với đó, `tcp_server_task` chờ kết nối từ PC. Khi nhận frame hợp lệ, task TCP cập nhật target RPM, hệ số PI, trạng thái enable PI hoặc yêu cầu PWM thủ công vào `g_cmd`. Khi PC yêu cầu đọc dữ liệu, task TCP lấy snapshot từ `g_tel` và đóng gói frame @REC gửi về.

Nhờ cấu trúc này, PC và Ethernet không còn nằm trong vòng điều khiển tốc độ. Nếu mạng bị trễ hoặc mất kết nối, task realtime vẫn chạy theo timer phần cứng và sẽ tự dừng motor khi timeout. Đây là cơ sở để hệ thống đạt tính realtime cục bộ tốt hơn so với kiến trúc điều khiển vòng kín trên PC.

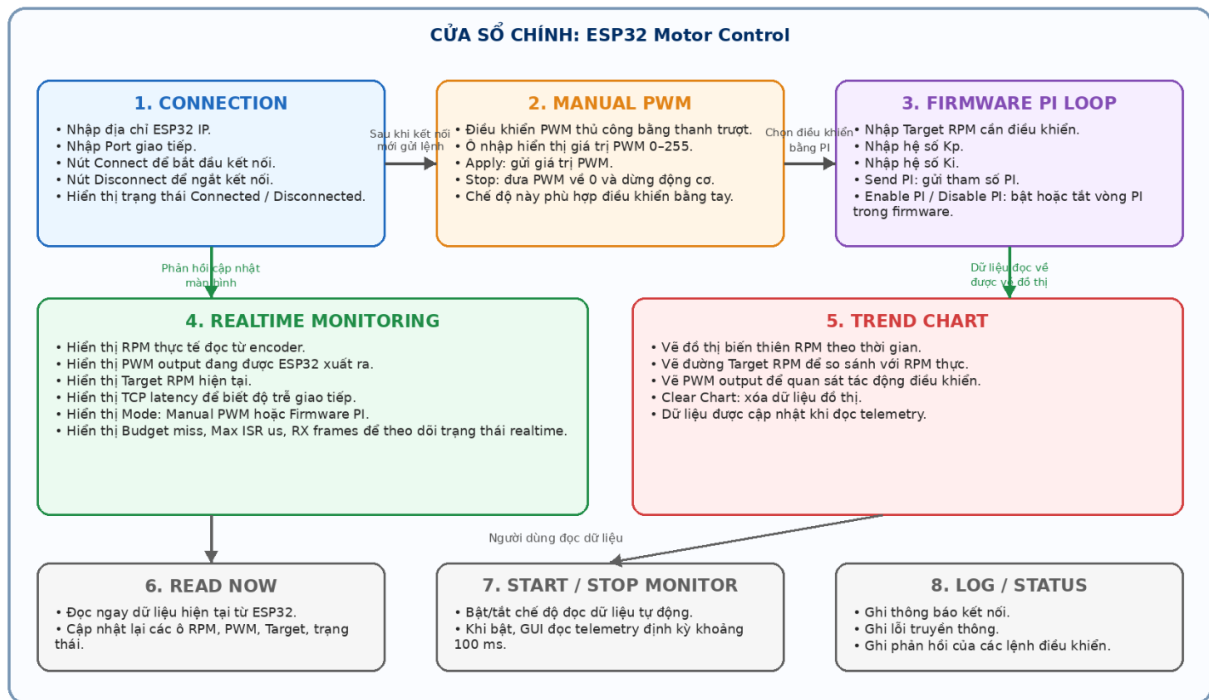
Chương 5

THIẾT KẾ GIAO DIỆN QUẢN LÝ BẰNG PYTHON

5.1 Tổng quan giao diện Python

Phần mềm giao diện trên máy tính được xây dựng bằng Python và thư viện Tkinter. Chương trình đóng vai trò TCP Client, kết nối đến TCP Server chạy trên ESP32 tại địa chỉ IP 192.168.2.177 và port 5000. Giao diện có nhiệm vụ gửi lệnh cấu hình, điều khiển thủ công PWM, cấu hình bộ điều khiển PI trong firmware và đọc dữ liệu giám sát realtime từ ESP32.

Trong kiến trúc tổng thể, Python GUI chỉ là HMI và công cụ giám sát. Vòng điều khiển tốc độ không chạy trên Python mà chạy trực tiếp trong firmware ESP32. Do đó, độ trễ của giao diện hoặc hệ điều hành máy tính không ảnh hưởng trực tiếp đến chu kỳ điều khiển motor.



Hình 5.1: Tổng quan giao diện Python Tkinter điều khiển và giám sát motor

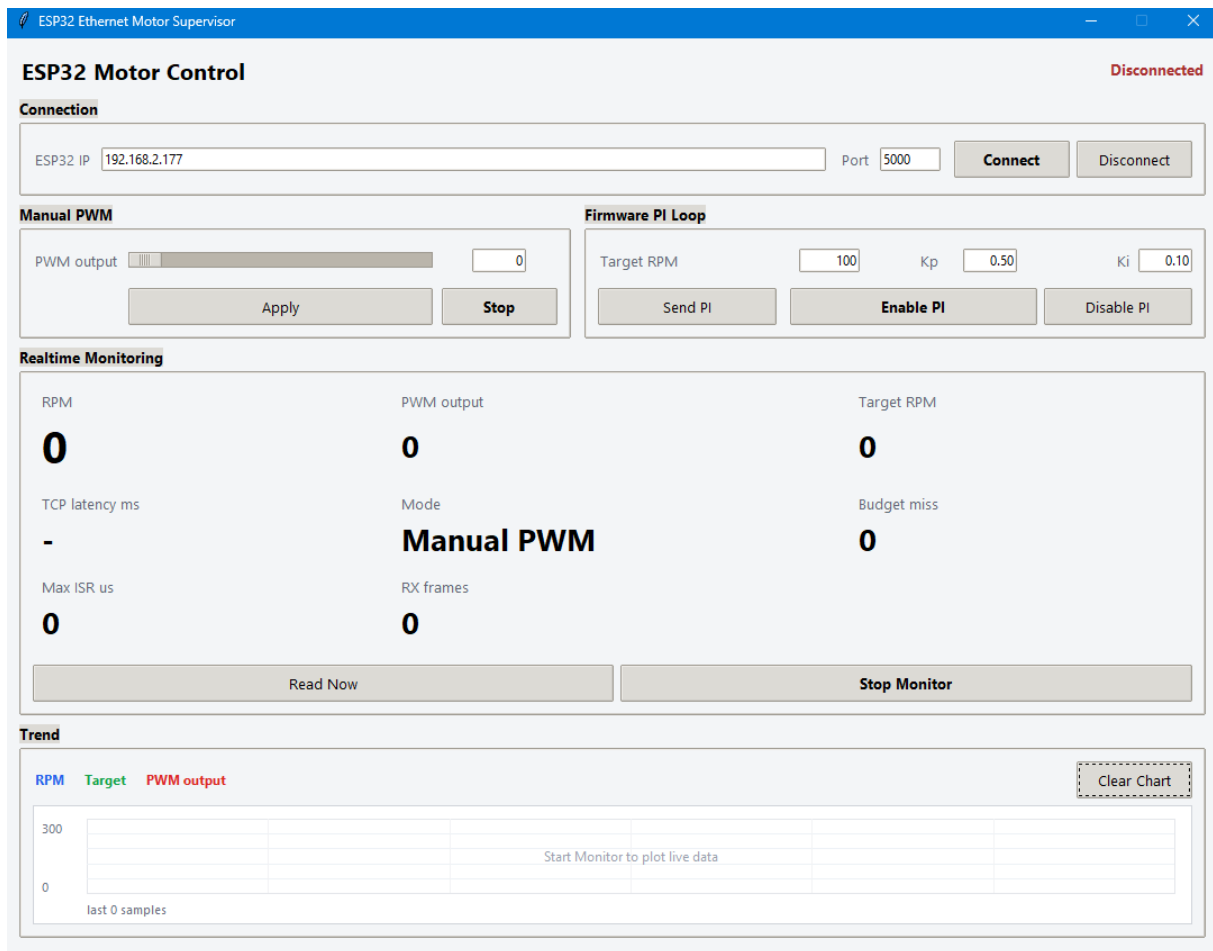
Các thư viện chính được sử dụng:

```

import queue
import socket
import struct
import threading
import time
import tkinter as tk
from dataclasses import dataclass
from tkinter import messagebox, ttk

```

Trong đó, `socket` dùng để giao tiếp TCP với ESP32, `threading` dùng để chạy luồng truyền thông riêng, `queue` dùng để trao đổi dữ liệu an toàn giữa luồng TCP và luồng giao diện, còn `tkinter/ttk` dùng để xây dựng cửa sổ điều khiển.



Hình 5.2: Giao diện được thiết kế

5.2 Định nghĩa giao thức và tham số giao tiếp

Chương trình Python sử dụng cùng bộ mã lệnh và cùng định dạng frame 12 byte với firmware ESP32. Các lệnh chính gồm ghi PWM thủ công, đọc RPM, đọc PWM hiện tại, đặt target RPM, đặt hệ số Kp, đặt hệ số Ki, bật/tắt PI và đọc các trường trạng thái.

```

CMD_WRITE_PWM = 0x02
CMD_STATUS = 0x04
CMD_READ_ENCODER = 0x07
CMD_SET_TARGET_RPM = 0x08
CMD_SET_KP = 0x09
CMD_SET_KI = 0x0A
CMD_PI_ENABLE = 0x0B
CMD_READ_PWM = 0x0C

```

```
STATUS_SEL_FLAGS = 0x00
STATUS_SEL_TARGET_RPM = 0x01
STATUS_SEL_DEADLINE_MISSES = 0x02
STATUS_SEL_MAX_RT_EXEC_US = 0x03
STATUS_SEL_RX_FRAMES = 0x04

PORT_DEFAULT = 5000
FRAME_LEN = 12
POLL_PERIOD_MS = 100
```

Địa chỉ mặc định của ESP32 trong giao diện là 192.168.2.177, khớp với IP tĩnh được cấu hình trong firmware. Chu kỳ đọc monitor mặc định là 100 ms, phù hợp với vai trò giám sát thay vì điều khiển realtime.

5.3 Tính CRC và đóng gói frame @SEN

Frame truyền từ Python xuống ESP32 có dạng @SEN, độ dài cố định 12 byte. Byte CRC được tính bằng phép XOR 10 byte đầu, giống như firmware.

```
def calc_crc(frame):
    crc = 0
    for i in range(10):
        crc ^= frame[i]
    return crc & 0xFF

def build_frame(cmd, value=0, seq=0, opt1=0, opt2=0):
    value &= 0xFFFF
    frame = bytearray(FRAME_LEN)
    frame[0:4] = b"@SEN"
    frame[4] = cmd & 0xFF
    frame[5] = (value >> 8) & 0xFF
    frame[6] = value & 0xFF
    frame[7] = opt1 & 0xFF
    frame[8] = opt2 & 0xFF
    frame[9] = seq & 0xFF
    frame[10] = calc_crc(frame)
    frame[11] = ord("#")
```



```
return bytes(frame)
```

Trường `value` được truyền bằng 2 byte theo thứ tự big-endian: byte cao ở vị trí 5 và byte thấp ở vị trí 6. Hai byte `opt1` và `opt2` dùng cho các tùy chọn phụ, ví dụ `opt1` được dùng làm selector khi đọc `CMD_STATUS`. Trường `seq` giúp kiểm tra phản hồi có đúng với lệnh vừa gửi hay không.

5.4 Nhận và giải mã frame @REC

Khi ESP32 phản hồi, Python nhận frame @REC cũng có độ dài 12 byte. Hàm `recv_exact()` đảm bảo nhận đủ số byte cần thiết, còn `parse_response()` kiểm tra header, ký tự kết thúc, CRC và số thứ tự frame.

```
def recv_exact(sock, size):
    buffer = bytearray(size)
    view = memoryview(buffer)
    received = 0
    while received < size:
        count = sock.recv_into(view[received:], size - received)
        if count == 0:
            raise ConnectionError("ESP32 closed the connection")
        received += count
    return bytes(buffer)

def parse_response(frame, expected_seq=None):
    if len(frame) != FRAME_LEN:
        raise ValueError("Response frame is not 12 bytes")
    if frame[0:4] != b"@REC" or frame[11] != ord("#"):
        raise ValueError("Invalid response frame header")
    if calc_crc(frame) != frame[10]:
        raise ValueError("Invalid response CRC")
    if expected_seq is not None and frame[9] != expected_seq:
        raise ValueError("Sequence mismatch")

    value = struct.unpack(">H", frame[5:7])[0]
    return {
        "cmd": frame[4],
```

```

    "value": value,
    "status": frame[7],
    "error": frame[8],
    "seq": frame[9],
}

```

Vì TCP là luồng byte, không phải gói tin rời rạc, việc dùng `recv_exact()` giúp chương trình không xử lý thiếu frame. Nếu ESP32 đóng kết nối hoặc frame sai định dạng, chương trình phát sinh lỗi để GUI hiển thị trong khung log.

5.5 Luồng TCP Client RealtimeClient

Để giao diện không bị treo khi truyền nhận TCP, chương trình tách phần giao tiếp mạng vào lớp `RealtimeClient`. Lớp này có một hàng đợi lệnh `command_queue` và một hàng đợi sự kiện `event_queue`. GUI đưa lệnh vào `command_queue`, còn luồng TCP xử lý lệnh và gửi kết quả về GUI bằng `event_queue`.

```

@dataclass
class CommandRequest:
    cmd: int
    value: int = 0
    label: str = ""
    callback: object = None
    opt1: int = 0
    opt2: int = 0

```

Cấu trúc `CommandRequest` lưu mã lệnh, dữ liệu 16 bit, nhãn hiển thị và các byte tùy chọn. Mỗi lần người dùng nhấn nút trên giao diện, một yêu cầu mới được đưa vào hàng đợi.

```

class RealtimeClient:
    def connect(self, host, port):
        self.disconnect()
        self.stop_event.clear()
        self.endpoint = (host, port)
        self.thread = threading.Thread(target=self._run, daemon=True)
        self.thread.start()

```

```
def send(self, cmd, value=0, label="", callback=None, opt1=0, opt2=0):
    self.command_queue.put(
        CommandRequest(cmd, value, label, callback, opt1, opt2)
    )
```

Hàm `connect()` tạo thread nền để xử lý TCP. Hàm `send()` không gửi socket trực tiếp trên luồng GUI, mà chỉ đưa lệnh vào queue. Cách này giúp thao tác trên giao diện luôn phản hồi nhanh.

```
def _run(self):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(2.0)
    sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
    sock.connect(self.endpoint)
    sock.settimeout(SOCKET_TIMEOUT_S)
    self.sock = sock
    self.connected = True
    self.event_queue.put(("connected", self.endpoint))

    while not self.stop_event.is_set():
        try:
            request = self.command_queue.get(timeout=0.05)
        except queue.Empty:
            continue

        started = time.perf_counter()
        response = self._transaction(
            request.cmd, request.value, request.opt1, request.opt2
        )
        latency_ms = (time.perf_counter() - started) * 1000.0
        self.event_queue.put(("response", request, response, latency_ms))
```

Trong mỗi transaction, chương trình tăng số seq, gửi frame @SEN, chờ đúng một frame @REC, sau đó đưa phản hồi về GUI. Thời gian transaction được tính thành `latency_ms` để hiển thị độ trễ TCP.

```
def _transaction(self, cmd, value, opt1=0, opt2=0):
    if self.sock is None:
        raise ConnectionError("Not connected")
```

```
self.seq = (self.seq + 1) & 0xFF
self.sock.sendall(build_frame(cmd, value, self.seq, opt1, opt2))
return parse_response(recv_exact(self.sock, FRAME_LEN), self.seq)
```

5.6 Thiết kế lớp giao diện MotorGUI

Lớp MotorGUI xây dựng toàn bộ cửa sổ điều khiển bằng Tkinter. Giao diện gồm các khối chính: kết nối TCP, điều khiển manual PWM, cấu hình PI trong firmware, realtime monitoring, biểu đồ xu hướng và khung log.

```
class MotorGUI:
    def __init__(self, root):
        self.root = root
        self.root.title("ESP32 Ethernet Motor Supervisor")
        self.root.geometry("1080x820")
        self.root.resizable(False, False)

        self.events = queue.Queue()
        self.client = RealtimeClient(self.events)
        self.auto_poll = False

        self.ip_var = tk.StringVar(value="192.168.2.177")
        self.port_var = tk.StringVar(value=str(PORT_DEFAULT))
        self.status_var = tk.StringVar(value="Disconnected")
        self.rpm_var = tk.StringVar(value="0")
        self.pwm_feedback_var = tk.StringVar(value="0")
        self.target_feedback_var = tk.StringVar(value="0")
```

Các biến StringVar và IntVar liên kết dữ liệu nội bộ với thành phần hiển thị trên GUI. Khi giá trị thay đổi, label hoặc entry tương ứng được cập nhật tự động.

5.7 Kết nối và ngắt kết nối ESP32

Khi người dùng nhấn nút Connect, GUI lấy địa chỉ IP và port từ ô nhập liệu, sau đó gọi RealtimeClient.connect(). Khi kết nối thành công, luồng TCP gửi sự kiện connected về GUI để cập nhật trạng thái.

```

def connect_esp32(self):
    try:
        self.client.connect(
            self.ip_var.get().strip(),
            int(self.port_var.get().strip())
        )
        self.status_var.set("Connecting...")
        self.status_label.configure(style="Warn.TLabel")
    except Exception as exc:
        messagebox.showerror("Connection error", str(exc))

def disconnect_esp32(self):
    self.auto_poll = False
    self.btn_auto.config(text="Start Monitor")
    self.client.disconnect()
    self.mode_var.set("Manual PWM")
    self.status_label.configure(style="Bad.TLabel")

```

Khi ngắt kết nối, chương trình dừng chế độ monitor tự động và đóng socket TCP. Ở phía firmware, nếu kết nối bị ngắt, ESP32 cũng có cơ chế dừng motor an toàn.

5.8 Điều khiển Manual PWM

Ở chế độ manual, người dùng chọn giá trị PWM từ 0 đến 255 bằng thanh trượt hoặc ô nhập số. Khi nhấn Apply, GUI gửi lệnh CMD_WRITE_PWM. Firmware nhận lệnh này, tắt PI và để task realtime áp dụng PWM thủ công.

```

def send_pwm(self):
    pwm = max(0, min(255, int(self.pwm_var.get())))
    self.pwm_var.set(pwm)
    self.pwm_scale.set(pwm)
    self.client.send(CMD_WRITE_PWM, pwm, "PWM")

def stop_motor(self):
    self.pwm_var.set(0)
    self.pwm_scale.set(0)
    self.client.send(CMD_WRITE_PWM, 0, "Stop")

```

```
self.client.send(CMD_PI_ENABLE, 0, "Disable PI")
```

Nút Stop gửi PWM bằng 0 và đồng thời gửi lệnh tắt PI. Đây là thao tác dừng mềm từ giao diện, khác với cơ chế timeout hoặc nút BOOT ở phía firmware.

5.9 Cấu hình bộ điều khiển PI trong firmware

Ở chế độ PI, Python không tự tính PWM. Giao diện chỉ gửi tốc độ đặt `target` RPM, hệ số `Kp`, `Ki` và lệnh bật PI xuống ESP32. Firmware sẽ tự đọc encoder, tính PI và cập nhật PWM trong `rt_control_task`.

```
def send_pi_params(self):
    try:
        target = max(0, min(65535, int(self.target_rpm_var.get())))
        kp_x100 = max(0, min(65535, int(float(self.kp_var.get()) * 100.0)))
        ki_x100 = max(0, min(65535, int(float(self.ki_var.get()) * 100.0)))
    except ValueError as exc:
        messagebox.showerror("PI parameter error", str(exc))
        return False

    self.client.send(CMD_SET_TARGET_RPM, target, "Target RPM")
    self.client.send(CMD_SET_KP, kp_x100, "Kp")
    self.client.send(CMD_SET_KI, ki_x100, "Ki")
    return True

def enable_pi(self):
    if self.send_pi_params():
        self.client.send(CMD_PI_ENABLE, 1, "Enable PI")

def disable_pi(self):
    self.client.send(CMD_PI_ENABLE, 0, "Disable PI")
```

Do firmware nhận `Kp` và `Ki` ở dạng số nguyên nhân 100, GUI chuyển giá trị thực nhập bởi người dùng sang `kp_x100` và `ki_x100`. Ví dụ, người dùng nhập 0.50 thì frame gửi xuống ESP32 có giá trị 50.

5.10 Đọc telemetry và monitor tự động

Chức năng realtime monitoring gửi nhiều lệnh đọc trạng thái xuống ESP32: đọc RPM, đọc PWM hiện tại, đọc cờ trạng thái, target RPM, số lần vượt deadline, thời gian thực thi realtime lớn nhất và số frame đã nhận.

```
def read_telemetry(self):
    self.client.send(CMD_READ_ENCODER, 0, "Read RPM")
    self.client.send(CMD_READ_PWM, 0, "Read PWM")
    self.client.send(CMD_STATUS, 0, "Read flags", opt1=STATUS_SEL_FLAGS)
    self.client.send(CMD_STATUS, 0, "Read target", opt1=STATUS_SEL_TARGET_RPM)
    self.client.send(CMD_STATUS, 0, "Read misses",
                     opt1=STATUS_SEL_DEADLINE_MISSES)
    self.client.send(CMD_STATUS, 0, "Read max ISR",
                     opt1=STATUS_SEL_MAX_RT_EXEC_US)
    self.client.send(CMD_STATUS, 0, "Read RX frames",
                     opt1=STATUS_SEL_RX_FRAMES)
```

Chế độ monitor tự động được thực hiện bằng hàm `_poll_telemetry()`. Hàm này được gọi định kỳ bởi `root.after()`, không dùng vòng lặp chặn nên giao diện vẫn hoạt động bình thường.

```
def toggle_auto_poll(self):
    self.auto_poll = not self.auto_poll
    if self.auto_poll:
        self.next_poll_deadline = time.perf_counter()
    self.btn_auto.config(
        text="Stop Monitor" if self.auto_poll else "Start Monitor"
    )

def _poll_telemetry(self):
    now = time.perf_counter()
    if self.auto_poll and self.client.connected and now >=
        self.next_poll_deadline:
        self.read_telemetry()
        while self.next_poll_deadline <= now:
            self.next_poll_deadline += POLL_PERIOD_S

    delay_s = max(0.001, self.next_poll_deadline - time.perf_counter())
```

```
self.root.after(max(1, int(delay_s * 1000)), self._poll_telemetry)
```

Chu kỳ đọc monitor là 100 ms. Đây là chu kỳ phục vụ hiển thị và ghi nhận trạng thái, không phải chu kỳ điều khiển motor.

5.11 Xử lý sự kiện từ luồng TCP

Tkinter yêu cầu cập nhật giao diện trên luồng chính. Vì vậy, luồng TCP không sửa trực tiếp label hoặc canvas, mà đưa sự kiện vào queue. Hàm `_drain_events()` được gọi định kỳ mỗi 16 ms để lấy sự kiện và cập nhật giao diện.

```
def _drain_events(self):
    while True:
        try:
            event = self.events.get_nowait()
        except queue.Empty:
            break
        self._handle_event(event)
    self.root.after(GUI_EVENT_PERIOD_MS, self._drain_events)

def _handle_event(self, event):
    kind = event[0]
    if kind == "connected":
        host, port = event[1]
        self.status_var.set(f"Connected to {host}:{port}")
        self.status_label.configure(style="Good.TLabel")
    elif kind == "error":
        self.log(f"I/O error: {event[1]}")
    elif kind == "response":
        _, request, response, latency_ms = event
        self.latency_var.set(f"{latency_ms:.1f}")
        if response["status"] != STATUS_OK:
            self.log(f"{request.label} failed, error={response['error']}")
            return
        self._apply_response(request, response)
```

Cơ chế queue giúp tránh lỗi cập nhật giao diện từ thread phụ và làm cho chương trình

ổn định hơn khi TCP bị mất kết nối hoặc phản hồi chậm.

5.12 Cập nhật dữ liệu phản hồi lên giao diện

Hàm `_apply_response()` giải mã phản hồi theo mã lệnh đã gửi. Nếu nhận RPM, GUI cập nhật label RPM và thêm mẫu vào đồ thị. Nếu nhận PWM, GUI cập nhật PWM feedback. Nếu nhận status, GUI cập nhật chế độ PI/manual, target RPM, deadline miss, thời gian thực thi lớn nhất hoặc số frame nhận.

```
def _apply_response(self, request, response):
    if request.cmd == CMD_READ_ENCODER:
        self.current_rpm = response["value"]
        self.rpm_var.set(str(self.current_rpm))
        self.append_chart_sample()
    elif request.cmd == CMD_READ_PWM:
        self.current_pwm = response["value"]
        self.pwm_feedback_var.set(str(self.current_pwm))
    elif request.cmd == CMD_WRITE_PWM:
        self.mode_var.set("Manual PWM")
        self.current_pwm = response["value"]
        self.pwm_feedback_var.set(str(self.current_pwm))
    elif request.cmd == CMD_PI_ENABLE:
        self.mode_var.set("Firmware PI" if response["value"] else "Manual PWM")
    elif request.cmd == CMD_STATUS:
        value = response["value"]
        if request.opt1 == STATUS_SEL_TARGET_RPM:
            self.current_target = value
            self.target_feedback_var.set(str(self.current_target))
```

Với lệnh đọc flags, GUI kiểm tra bit `CONTROL_FLAG_PI_ENABLED` để xác định firmware đang chạy chế độ PI hay manual PWM. Nhờ đó trạng thái hiển thị phản ánh đúng mode thực tế trong ESP32.

5.13 Vẽ biểu đồ realtime

Giao diện sử dụng `Canvas` để vẽ đồ thị xu hướng gồm ba đường: RPM thực tế, target RPM và PWM output. Dữ liệu biểu đồ được lưu trong danh sách `chart_samples`, giới hạn tối đa 600 điểm để tránh tốn bộ nhớ khi chạy lâu.

```
def append_chart_sample(self):
    self.chart_samples.append(
        (time.perf_counter(), self.current_rpm,
         self.current_target, self.current_pwm)
    )
    if len(self.chart_samples) > CHART_HISTORY_POINTS:
        del self.chart_samples[
            : len(self.chart_samples) - CHART_HISTORY_POINTS
        ]
    self.draw_chart()
```

Hàm `draw_chart()` xóa canvas cũ, vẽ lưới nền, tự động chọn thang RPM theo giá trị lớn nhất và vẽ ba đường dữ liệu với màu khác nhau. Đường RPM giúp quan sát tốc độ thực tế, đường target giúp so sánh khả năng bám tốc độ, còn đường PWM cho biết mức điều khiển mà firmware đang xuất ra.

```
canvas.create_line(*points(1, rpm_max), fill="#2563eb", width=2, smooth=True)
canvas.create_line(*points(2, rpm_max), fill="#16a34a", width=2, smooth=True)
canvas.create_line(*points(3, CHART_PWM_MAX), fill="#dc2626", width=2,
    smooth=True)
```

5.14 Ghi log và đóng chương trình

Khung log dùng để ghi lại các sự kiện quan trọng như kết nối thành công, lỗi I/O, PWM đã áp dụng, PI bật/tắt và các lỗi phản hồi từ firmware. Mỗi dòng log có thời gian để thuận tiện kiểm tra quá trình vận hành.

```
def log(self, msg):
    stamp = time.strftime("%H:%M:%S")
    self.log_text.insert("end", f"{stamp} {msg}\n")
    self.log_text.see("end")
```

```
def on_close(self):  
    self.disconnect_esp32()  
    self.root.destroy()
```

Khi người dùng đóng cửa sổ, chương trình chủ động ngắt kết nối TCP trước khi hủy giao diện. Điều này giúp socket được giải phóng đúng cách và firmware có thể phát hiện mất kết nối để dừng motor an toàn nếu cần.

Chương 6

TỔNG KẾT

6.1 Kết quả đạt được

Đề tài đã xây dựng được hệ thống giám sát và điều khiển tốc độ động cơ DC qua Ethernet sử dụng ESP32 kết hợp module W5500. ESP32 đóng vai trò thiết bị nhúng điều khiển cục bộ, còn máy tính đóng vai trò giao diện HMI để cấu hình, giám sát và ghi log dữ liệu. Hệ thống sử dụng TCP/IP trong mạng LAN để truyền nhận dữ liệu giữa phần mềm Python và firmware ESP32.

Về phần firmware, chương trình đã được tổ chức theo kiến trúc hai core của ESP32. Core 0 chạy `tcp_server_task` để nhận lệnh từ máy tính, kiểm tra frame, cập nhật bộ đếm lệnh và gửi phản hồi. Core 1 chạy `rt_control_task` với độ ưu tiên cao để thực hiện vòng điều khiển thời gian thực: đọc encoder, tính RPM, xử lý manual PWM hoặc PI, cập nhật PWM và ghi telemetry. Nhờ đó, Ethernet TCP không nằm trực tiếp trong vòng điều khiển motor.

Hệ thống đã xây dựng giao thức truyền thông ứng dụng dạng frame cố định 12 byte. Frame gửi từ máy tính xuống ESP32 có header `@SEN`, frame phản hồi từ ESP32 về máy tính có header `@REC`. Mỗi frame gồm mã lệnh, dữ liệu 16 bit, các byte tùy chọn, số thứ tự frame và checksum XOR. Cấu trúc này giúp quá trình truyền nhận có định dạng rõ ràng, dễ kiểm tra lỗi và dễ mở rộng thêm lệnh điều khiển.

Về phần giao diện, phần mềm Python Tkinter được thiết kế đúng vai trò HMI. Giao diện có các chức năng kết nối/ngắt kết nối TCP, điều khiển PWM thủ công, đặt target RPM, gửi hệ số K_p/K_i , bật/tắt PI trong firmware, đọc telemetry, hiển thị trạng thái realtime, ghi log và vẽ biểu đồ RPM/PWM. Luồng giao tiếp TCP được tách khỏi luồng giao diện bằng `threading` và `queue`, giúp giao diện không bị treo khi truyền nhận dữ liệu.

6.2 Ý nghĩa của kiến trúc realtime cục bộ

Một kết quả quan trọng của đề tài là hệ thống không sử dụng máy tính để đóng vòng điều khiển tốc độ. Máy tính chỉ gửi tham số và đọc trạng thái, còn ESP32 tự thực hiện vòng điều khiển bằng timer phần cứng và task realtime. Cách tổ chức này phù hợp hơn với yêu cầu điều khiển thực tế, vì jitter của TCP, hệ điều hành máy tính hoặc giao diện Python không ảnh hưởng trực tiếp đến chu kỳ điều khiển motor.

Firmware cũng có các cơ chế an toàn cơ bản như timeout truyền thông, dừng motor khi mất kết nối TCP, nút BOOT để yêu cầu dừng motor và telemetry để theo dõi số lần vượt deadline. Các dữ liệu như RPM, PWM, target RPM, trạng thái PI, số frame nhận và thời gian thực thi realtime lớn nhất giúp người dùng đánh giá hoạt động của hệ thống trong quá trình thử nghiệm.

6.3 Hạn chế

Hệ thống hiện tại vẫn là mô hình thực nghiệm, chưa phải một bộ điều khiển công nghiệp hoàn chỉnh. Checksum đang dùng XOR 1 byte nên chỉ phù hợp cho mức demo và kiểm tra lỗi đơn giản. Frame truyền thông cố định 12 byte giúp hệ thống dễ xử lý nhưng giới hạn lượng dữ liệu telemetry trong mỗi phản hồi. Ngoài ra, TCP không phải giao thức realtime công nghiệp, vì vậy chỉ phù hợp cho vai trò cấu hình và giám sát, không nên dùng để đóng vòng điều khiển tốc độ từ PC.

Encoder hiện tại chỉ đọc một kênh xung nên hệ thống mới xác định tốc độ quay, chưa xác định được chiều quay. Bộ điều khiển đang dùng PI với tham số nhập thủ công, chưa có cơ chế tự tuning hoặc nhận dạng mô hình động cơ. Giao diện Python phục vụ tốt cho giám sát và thử nghiệm, nhưng chưa có các chức năng nâng cao như lưu file log dài hạn, reset fault riêng, hiển thị fault chi tiết hoặc xuất dữ liệu sang Matlab/Simulink.

6.4 Hướng phát triển

Trong các phiên bản tiếp theo, hệ thống có thể được nâng cấp theo một số hướng. Về firmware, có thể bổ sung CRC-16 thay cho checksum XOR, mở rộng frame telemetry, thêm timestamp hoặc cycle counter, bổ sung fault state rõ ràng và thêm lệnh reset fault. Vòng điều khiển có thể được cải tiến bằng cách đọc encoder hai kênh để xác định chiều quay, thêm giới hạn an toàn cho tốc độ, dòng điện hoặc lỗi encoder.

Về giao diện, phần mềm Python có thể bổ sung chức năng lưu dữ liệu ra file CSV, vẽ đồ thị dài hạn, hiển thị fault flags theo tên lỗi, thêm nút emergency stop và reset fault. Nếu cần kết nối với Matlab/Simulink, Python hoặc ESP32 có thể xuất dữ liệu telemetry theo định dạng chuẩn hơn để phục vụ phân tích đáp ứng tốc độ và chỉnh định tham số PI.

Nhìn chung, đề tài đã kết hợp được các nội dung quan trọng gồm truyền thông Ethernet, mô hình TCP/IP, lập trình ESP32, giao tiếp SPI với W5500, thiết kế giao thức frame, lập trình giao diện Python và điều khiển tốc độ động cơ DC có encoder. Kiến trúc cuối cùng phù hợp với định hướng realtime cục bộ: ESP32 tự điều khiển motor, còn máy tính đảm nhiệm cấu hình và giám sát.

Tài liệu tham khảo

- [1] Espressif Systems, *ESP32 Series Datasheet*. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [2] Espressif Systems, *ESP-IDF Programming Guide – Ethernet Driver*. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_eth.html
- [3] Espressif Systems, *ESP-IDF Programming Guide – LED Control (LEDC)*. Available: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/ledc.html>
- [4] Espressif Systems, *ESP-IDF Programming Guide – Pulse Counter (PCNT)*. Available: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/pcnt.html>
- [5] WIZnet Co., Ltd., *W5500 Datasheet*. Available: <https://docs.wiznet.io/Product/iEthernet/W5500/datasheet>
- [6] Python Software Foundation, *socket – Low-level networking interface*. Available: <https://docs.python.org/3/library/socket.html>
- [7] Python Software Foundation, *tkinter – Python interface to Tcl/Tk*. Available: <https://docs.python.org/3/library/tkinter.html>
- [8] Python Software Foundation, *threading – Thread-based parallelism*. Available: <https://docs.python.org/3/library/threading.html>
- [9] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Pearson, 2021.
- [10] MathWorks, *TCP/IP Communication and Instrument Control Documentation*. Available: <https://www.mathworks.com/help/>